

Betriebssysteme

Kapitel 6: Speicherverwaltung

Prof. Dr. Raphael Herding

Inhalte des Kapitels (1)

6. Speicherverwaltung

- Einleitung
 - Phänomen der Speicherverwaltung
 - Speicherhierarchien
 - Kapazität von Speichern
- Speichervergabe
 - Statische und dynamische Speicherzuteilung
 - Freispeicherverwaltung (Bitvektoren, verkettete Listen)
 - Vergabestrategien
 - *First Fit*
 - *Next Fit*
 - *Best Fit*
 - *Worst Fit*
- ...

Inhalte des Kapitels (2)

6. Speicherverwaltung

- ...
- Mehrprogrammbetrieb
 - Segmentierung
 - Kompaktierung
 - Seitenadressierung (*Paging*)
 - *Memory Management Unit* für *Paging*
 - Segmentierung und Seitenadressierung
 - Mehrstufiges *Paging*
 - *Translation Look-Aside Buffer (TLBs)*
- ...

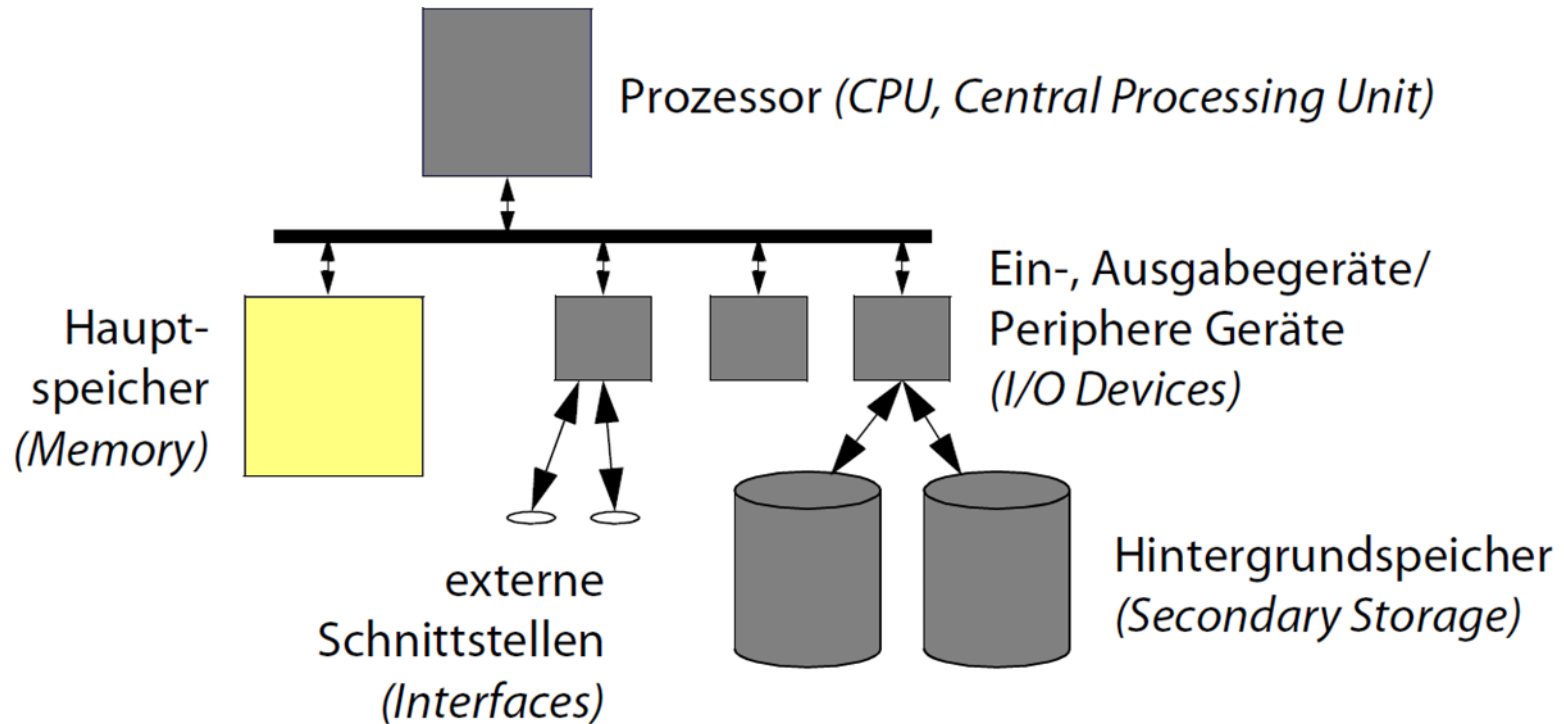
Inhalte des Kapitels (3)

6. Speicherverwaltung

- ...
- Virtueller Speicher
 - *Demand Paging*
 - Seiten-Ersetzungsstrategien
 - *First-In First-Out (FIFO)*
 - Optimale Ersetzungsstrategie
 - *Least Recently Used (LRU)*
 - *Second Chance (Clock)*
 - Freiseitenpuffer
 - Seitenadressierung und Mehrprogrammbetrieb

Einordnung

Betroffene physikalische Ressourcen



Phänomen der Speicherverwaltung (1)

Beispiel: Initialisierung großer Matrizen (geschrieben in C)

– Variante 1

```
#define DIM 10000

int main()
{
    register long i, j;
    static long matrix[DIM][DIM];

    for ( i = 0; i < DIM; i++ )
        for ( j = 0; j < DIM; j++ )
            matrix[i][j] = 1;

    return 0;
}
```

Phänomen der Speicherverwaltung (2)

Beispiel: Initialisierung großer Matrizen (geschrieben in C)

- Variante 2

```
#define DIM 10000

int main()
{
    register long i, j;
    static long matrix[DIM][DIM];

    for ( j = 0; j < DIM; j++ )
        for ( i = 0; i < DIM; i++ )
            matrix[i][j] = 1;

    return 0;
}
```

- Schleifen sind vertauscht

Phänomen der Speicherverwaltung (3)

Messergebnisse (Intel Xeon 2,4 GHz, 2GB Hauptspeicher)

- Variante 1

User time = 0,26 sec; System time = 0,83 sec; Gesamtzeit = 1,09 sec

- Variante 2

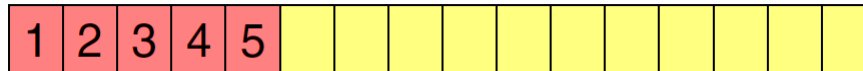
User time = 17 sec; System time = 0,81 sec; Gesamtzeit = 17,9 sec

Ursachen

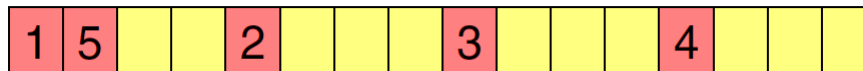
- Variante 1 geht sequentiell durch den Speicher
- Variante 2 greift versetzt ständig auf den gesamten Speicherbereich zu

Beispiel: `matrix[4][4]` und die ersten fünf Zugriffe

Variante 1



Variante 2



Phänomen der Speicherverwaltung (4)

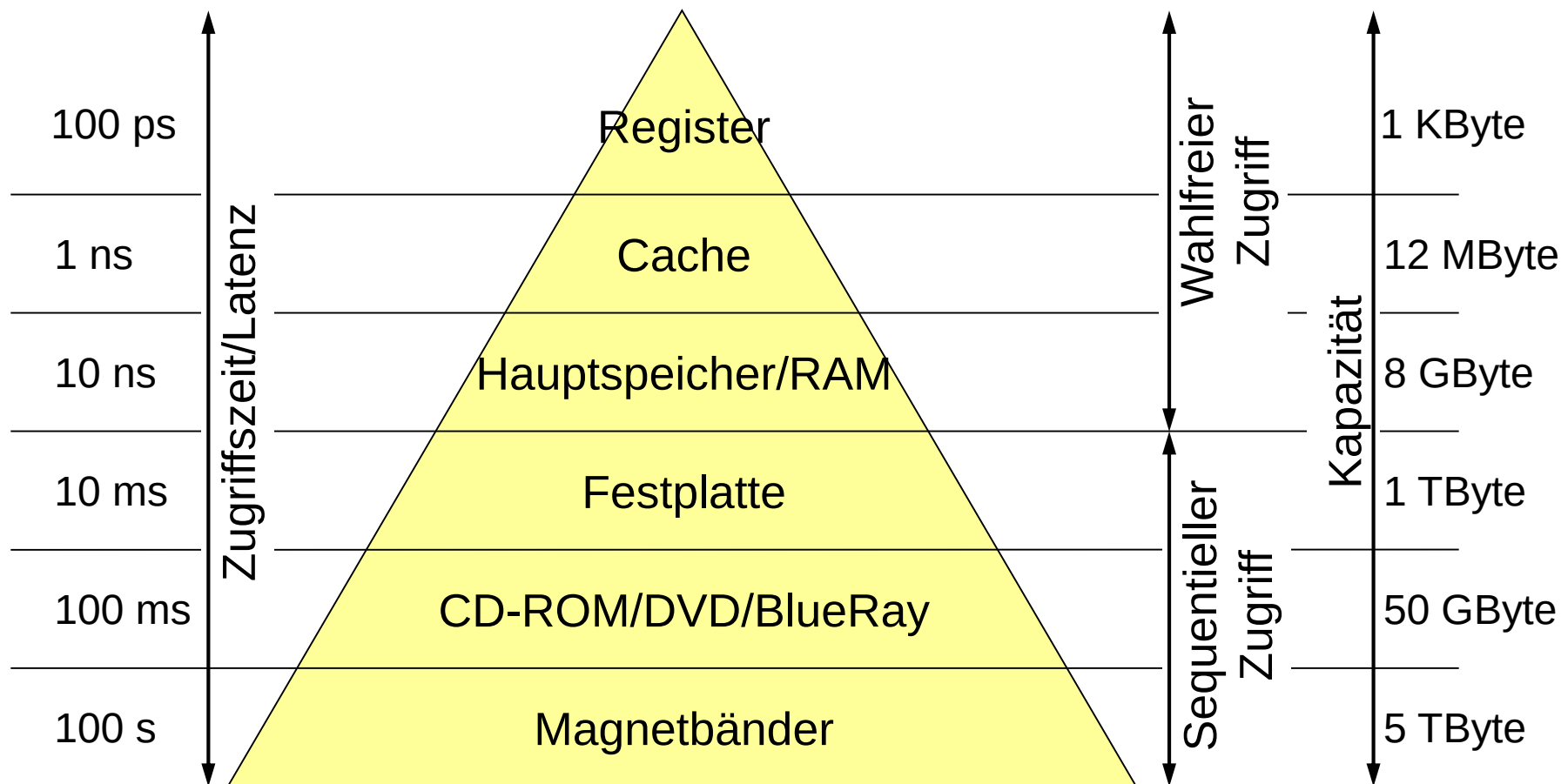
Ursachen

- Virtueller (logischer) Adressraum
 - Benutzte Adressen sind nicht die realen (physikalischen) Adressen
 - Abbildung wird durch Hardware auf Seitenbasis vorgenommen (Seitenadressierung)
 - Variante 2 hat weniger Lokalität, d.h. benötigt häufig wechselnde Abbildungen
- Virtueller Speicher
 - Möglicher Adressraum ist größer als realer Speicher
 - Auf Seitenbasis werden Teile des benötigten Speichers ein- und ausgelagert
 - Bei Variante 2 muss viel mehr Speicher ein- und ausgelagert werden

Speicher allgemein (1)

Speicherhierarchie

- Bis zu sechs Ebenen in modernen Systemen unterscheidbar



Speicher allgemein (2)

Messung der Kapazität von Speicherbausteinen in

- *Kilobyte (kByte, kB)*

$$1 \text{ kByte} = 1024 \text{ Bytes} = 2^{10} \text{ Bytes}$$

- *Megabyte (MByte, MB)*

$$1 \text{ MByte} = 1024 \text{ kBytes} = 2^{20} \text{ Bytes} = 1.048.576 \text{ Bytes}$$

- *Gigabyte (GByte, GB)*

$$1 \text{ GByte} = 1024 \text{ MBytes} = 2^{30} \text{ Bytes} = 1.073.741.824 \text{ Bytes}$$

- *Terabyte (TByte, TB)*

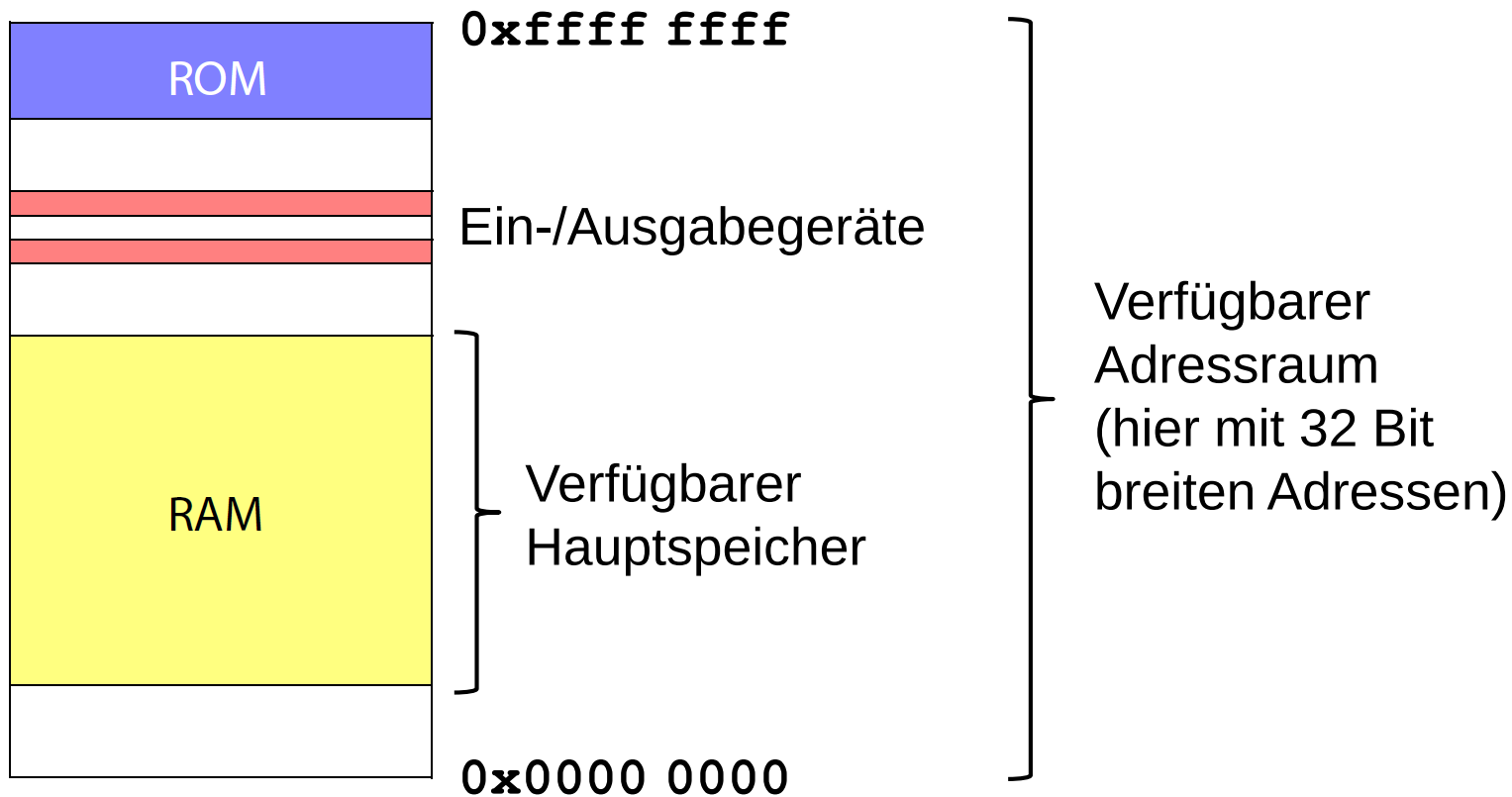
$$1 \text{ TByte} = 1024 \text{ GBytes} = 2^{40} \text{ Bytes} = 1.099.511.627.776 \text{ Bytes}$$

- *Petabyte (PByte, PB)*

$$1 \text{ PByte} = 1024 \text{ TBytes} = 2^{50} \text{ Bytes} = 1.125.899.906.842.624 \text{ Bytes}$$

Speichervergabe (1)

Verfügbarer Speicher



Speichervergabe (2)

Belegung des verfügbaren Hauptspeichers durch

- Benutzerprogramme
 - Programmbefehle (Code, *Binary*)
 - Programmdateien
- Betriebssystem
 - Betriebssystemcode
 - Pufferspeicher
 - Systemvariablen

☞ Zuteilung des Speichers nötig

Statische Speicherzuteilung

Feste Bereiche für Betriebssystem und Benutzerprogramm

Probleme

- Begrenzung anderer Ressourcen
(z.B. Bandbreite bei Ein-/Ausgabe wegen zu kleiner Systempuffer)
- Ungenutzter Speicher des Betriebssystems kann von Anwendungsprogramm nicht genutzt werden, und umgekehrt

☞ **Dynamische Speicherzuteilung einsetzen**

Dynamische Speicherzuteilung

Segmente

- Zusammenhängender Speicherbereich
(Bereich mit aufeinanderfolgenden Adressen)
- Einzelne Segmente können unterschiedliche Länge haben

Erforderlich: Allokation (Anforderung) und Freigabe von Segmenten

Ein Anwendungsprogramm besitzt üblicherweise folgende Segmente

- Code-Segment
- Daten-Segment
- *Stack*-Segment (für Verwaltungsinformationen, z.B. bei Funktionsaufrufen)

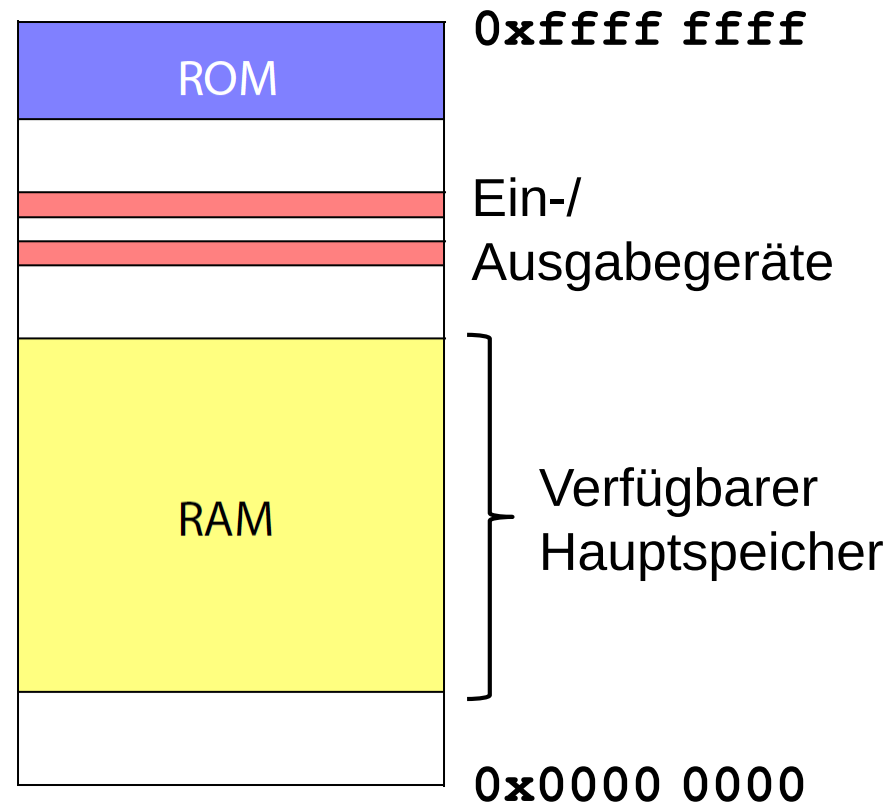
☞ **Suche nach geeigneten Speicherbereichen zur Zuteilung**

☞ **Speicherzuteilungsstrategien**

Adressräume (1)

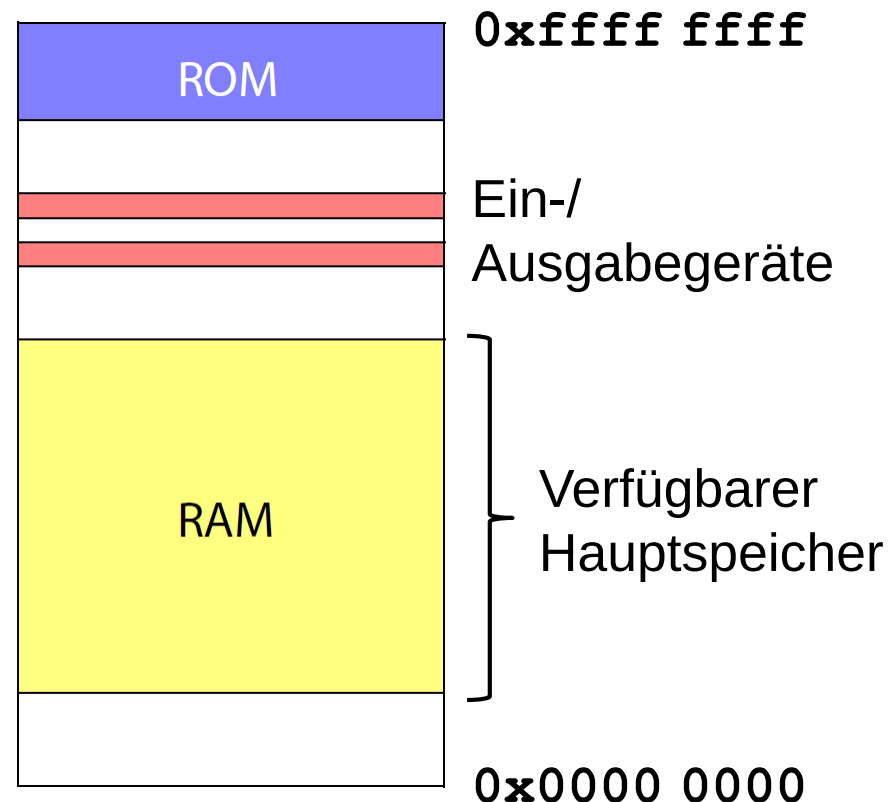
Physikalischer Adressraum

- Eine 32 Bit-Architektur kann physikalische Adressen von 0, ..., $2^{32}-1$ erzeugen
- Nicht alle diese physikalischen Adressen gibt es in Hardware (weiße Bereiche rechts)
- Einzelne Teilbereiche des phys. Adressraums werden auf unterschiedliche Hardware-Komponenten abgebildet (bspw. DRAM-Module, Boot-ROMs, Video-Speicher, ...)



Adressräume (1)

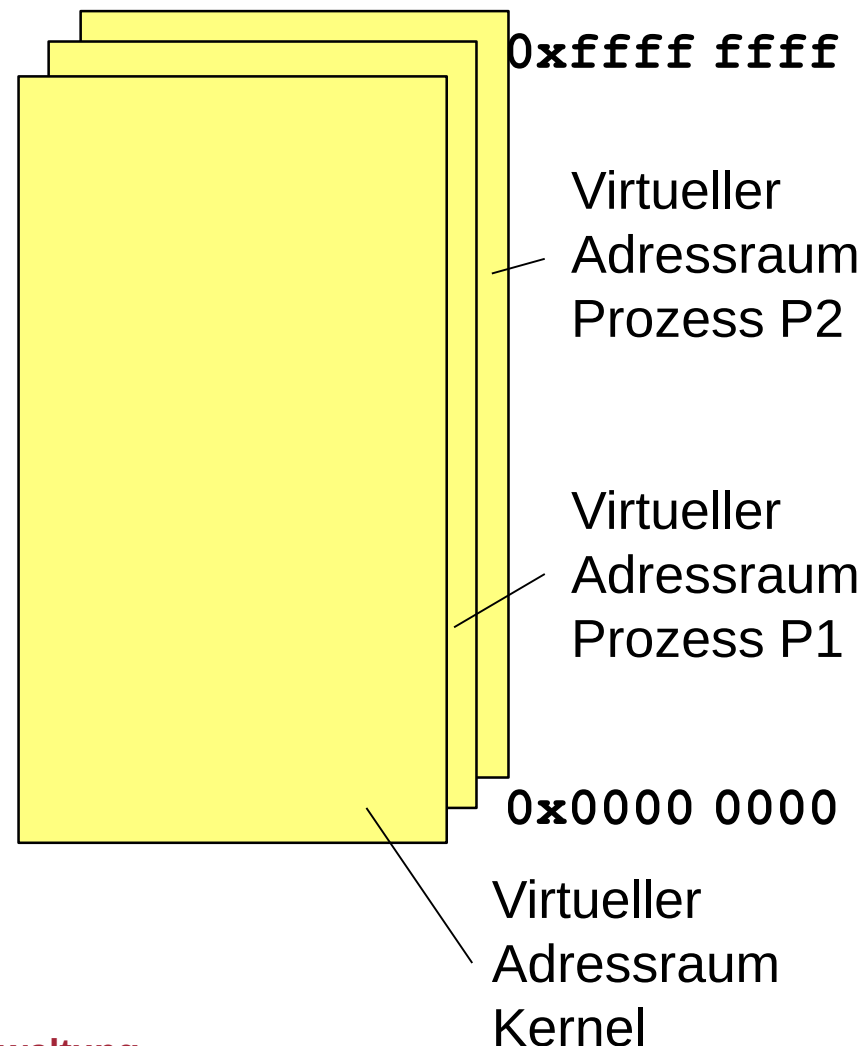
- **32 Bit- vs. 64Bit-Architektur**
- Anzahl Bits geben physikalische Grenze des Speichers an.
- 32 Bit ermöglicht Adressen von 0, ..., $2^{32}-1$
- 64 Bit ermöglicht Adressen von 0, ..., $2^{64}-1$
- Zahlenrepräsentationen sind unangetastet (z.B. int ist 4 Byte).
- Adresse zur Speicherstelle ist jedoch nun 64 Bit.



Adressräume (2)

Virtueller Adressraum

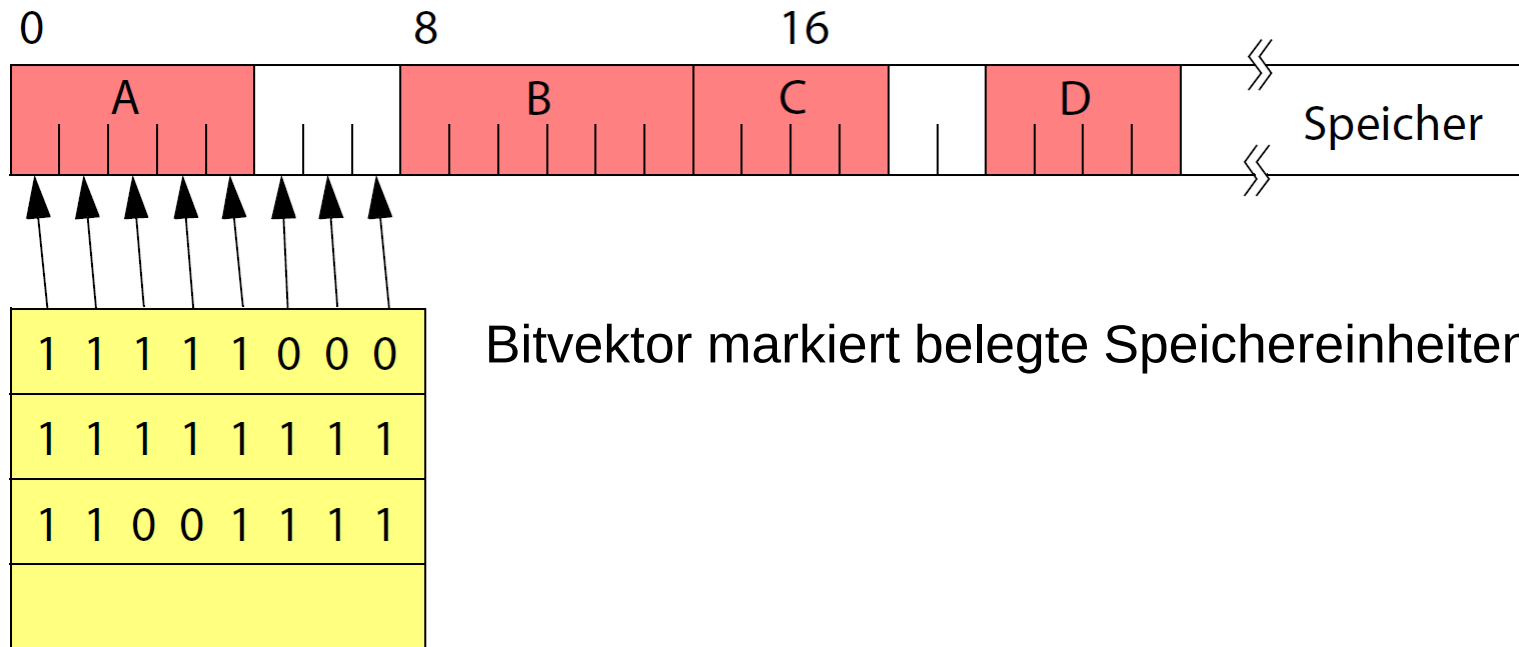
- Eine Software kann viel mehr Adressen benutzen als in physikalischem Adressraum überhaupt existieren
- Software wird ein virtueller Adressraum durch das Betriebssystem „vorgespiegelt“, der sämtliche möglichen Adressen $0, \dots, 2^{32}-1$ umfassen kann
- Jeder einzelne Anwender-Prozess hat seinen eigenen virtuellen Adressraum (*user space*), und auch das Betriebssystem (*kernel space*)



Freispeicherverwaltung (1)

Freie (evtl. auch belegte) Segmente des Speichers müssen repräsentiert werden

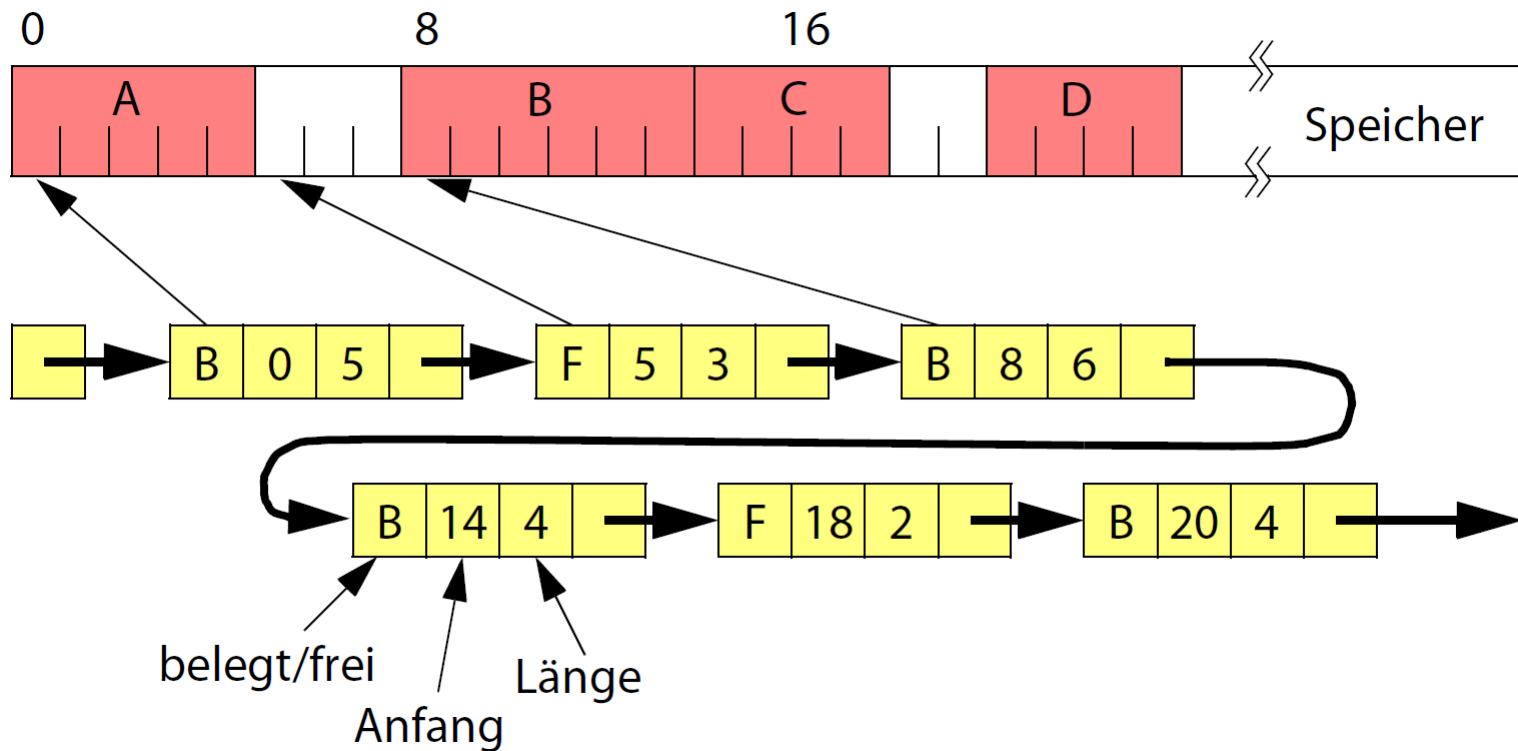
Bitvektoren



Speichereinheiten gleicher Größe (z.B. 1 Byte, 64 Bytes, 1 kByte)

Freispeicherverwaltung (2)

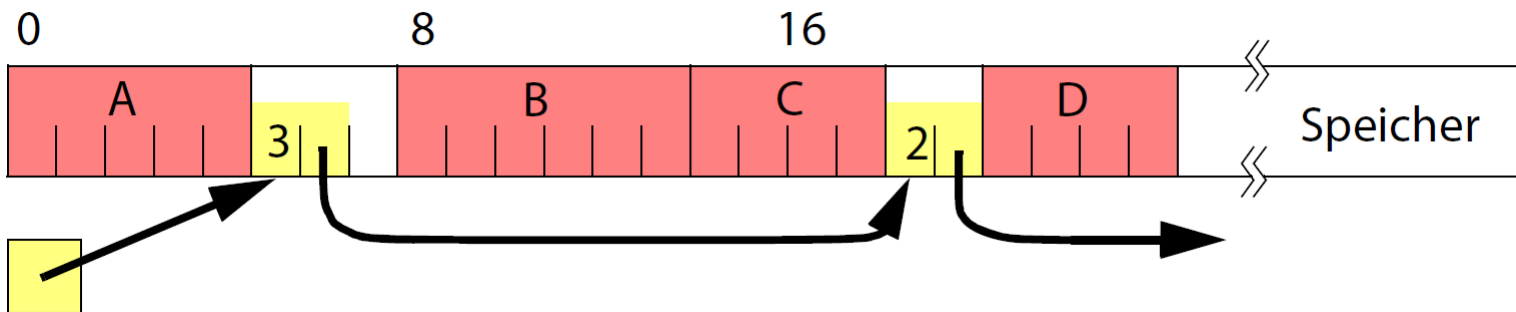
Verkettete Liste



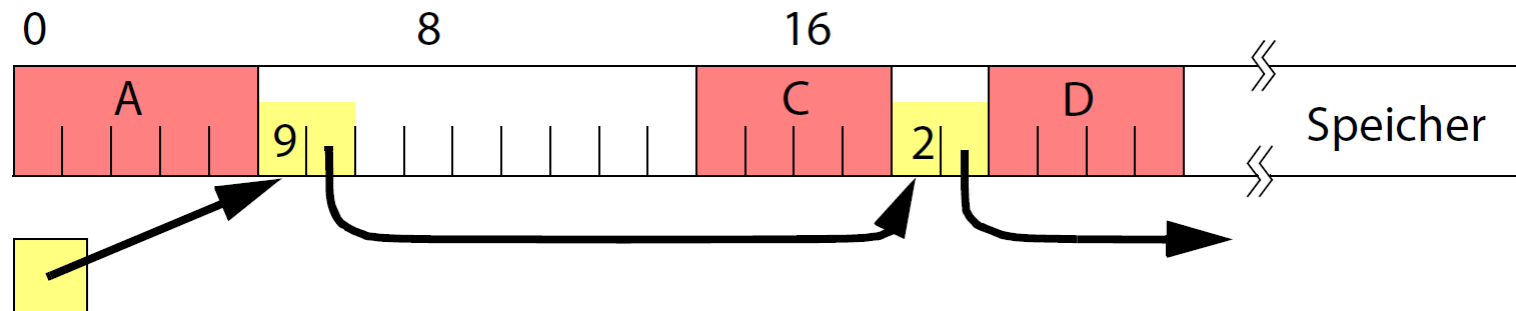
Repräsentation auch von freien Segmenten

Speichervergabe

Verschmelzung von Lücken



nach Freigabe von Segment B:



Einsatz der Verfahren

Einsatz im Betriebssystem

- Verwaltung des Systemspeichers
- Zuteilung von Speicher an Prozesse und Betriebssystem

Einsatz innerhalb eines Prozesses

- Verwaltung des Heapspeichers (*Heap*)
- Erlaubt dynamische Allokation von Speicherbereichen durch den Prozess (**malloc** und **free**)

Einsatz für Bereiche des Sekundärspeichers

- Verwaltung bestimmter Abschnitte des Sekundärspeichers (Festplatte)
 - Z.B. Speicherbereich für Prozessauslagerungen (*swap space*)

Roter Faden

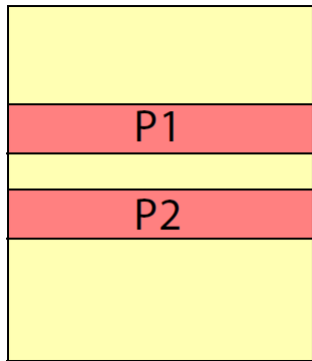
6. Speicherverwaltung

- Einleitung
- Speichervergabe
 - Statische und dynamische Speicherzuteilung
 - Freispeicherverwaltung (Bitvektoren, verkettete Listen)
- Mehrprogrammbetrieb
- Virtueller Speicher

Mehrprogrammbetrieb

Problemstellung: Mehrere Prozesse benötigen Hauptspeicher

- Prozesse an verschiedenen Stellen im Hauptspeicher



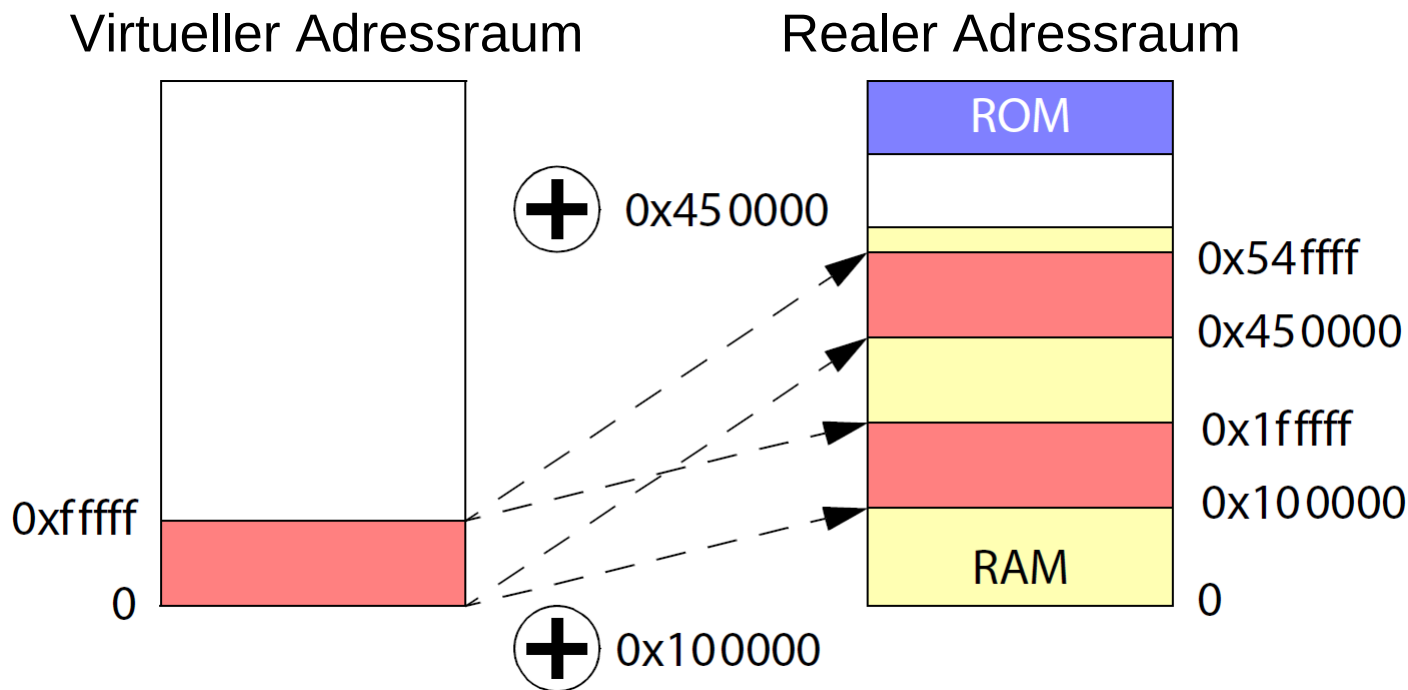
Zwei Prozesse und deren Code-Segmente im Speicher

- Hauptspeicher nicht ausreichend
- Speicherschutz zwischen Betriebssystem und Prozessen sowie zwischen Prozessen untereinander

Segmentierung (1)

Hardwareunterstützung: Umsetzung virtueller in reale Adressen

- Prozesse erhalten einen virtuellen Adressraum



Das Segment des virtuellen Adressraums kann an jeder beliebigen Stelle im realen Adressraum liegen

Segmentierung (4)

Hardware zur Adress-Umsetzung: *Memory Management Unit (MMU)*

👍 **Schutz vor Segmentübertretung**

- Unterbrechung zeigt Speicherverletzung an
- Programme und Betriebssystem voreinander geschützt

👍 **Kontextwechsel durch Austausch der Segmentbasis**

- Jeder Prozess hat eigene Übersetzungstabelle (gehört zum Kontext)

👍 **Ein- und Auslagerung vereinfacht**

- Nach Einlagerung an beliebige Stelle muss lediglich die Übersetzungstabelle angepasst werden

👍 **Gemeinsame Segmente möglich**

- Code-Segmente (Befehlssegmente)
- Datensegmente (*shared memory*)

Segmentierung (5)

👍 Zugriffsschutz einfach integrierbar

- Z.B. Rechte zum Lesen, Schreiben und Ausführen von Befehlen, die von der MMU geprüft werden

👎 Externe Fragmentierung des Speichers durch häufiges Ein- und Auslagern

- Es entstehen kleine, nicht nutzbare Lücken
- Lücken sind außerhalb (extern) der Segmente im Freispeicher-Bereich

👎 Kompaktierung

- Segmente werden verschoben, um Lücken zu schließen
- Segmenttabelle wird jeweils angepasst

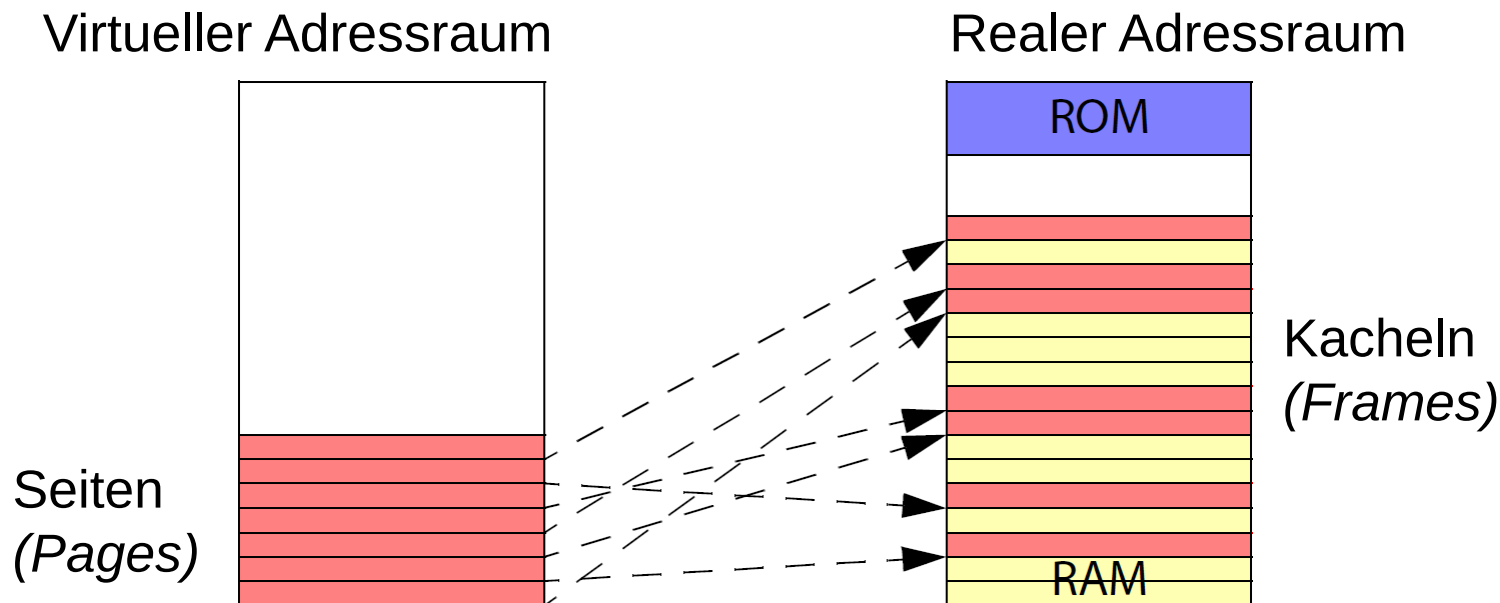
👎 Lange I/O-Zeiten für Ein- und Auslagerung

- Nicht alle Teile eines Segments werden gleich häufig genutzt

Seitenadressierung (*Paging*)

Einteilung des virtuellen Adressraums in gleichgroße Seiten, die an beliebigen Stellen im realen Adressraum liegen können

- Lösung des Problems der externen Fragmentierung
- Vereinfacht Speicherbelegung und Ein-/Auslagerungen



Ein- und Auslagerung von Seiten

Es ist nicht nötig, ein gesamtes Segment aus- bzw. einzulagern

- Seiten können einzeln ein- und ausgelagert werden

Hardware-Unterstützung

	Startadr.	Präsenzbit
0000		
0001		
0002	ffe0 fxxx	X
	...	

- Ist das Präsenzbit gesetzt, bleibt alles wie bisher
- Ist das Präsenzbit gelöscht, wird eine Unterbrechung ausgelöst (*page fault*)
- Die Unterbrechungsbehandlung kann nun für das Laden der Seite vom Hintergrundspeicher sorgen und den Speicherzugriff danach wiederholen
(benötigt HW-Unterstützung in der CPU)

Roter Faden

6. Speicherverwaltung

- Einleitung
- Speichervergabe
- Mehrprogrammbetrieb
 - Segmentierung
 - Kompaktierung
 - Seitenadressierung (*Paging*)
 - *Memory Management Unit* für *Paging*
 - Segmentierung und Seitenadressierung
- Virtueller Speicher

Virtueller Speicher

Entkopplung des Speicherbedarfs vom verfügbaren Hauptspeicher

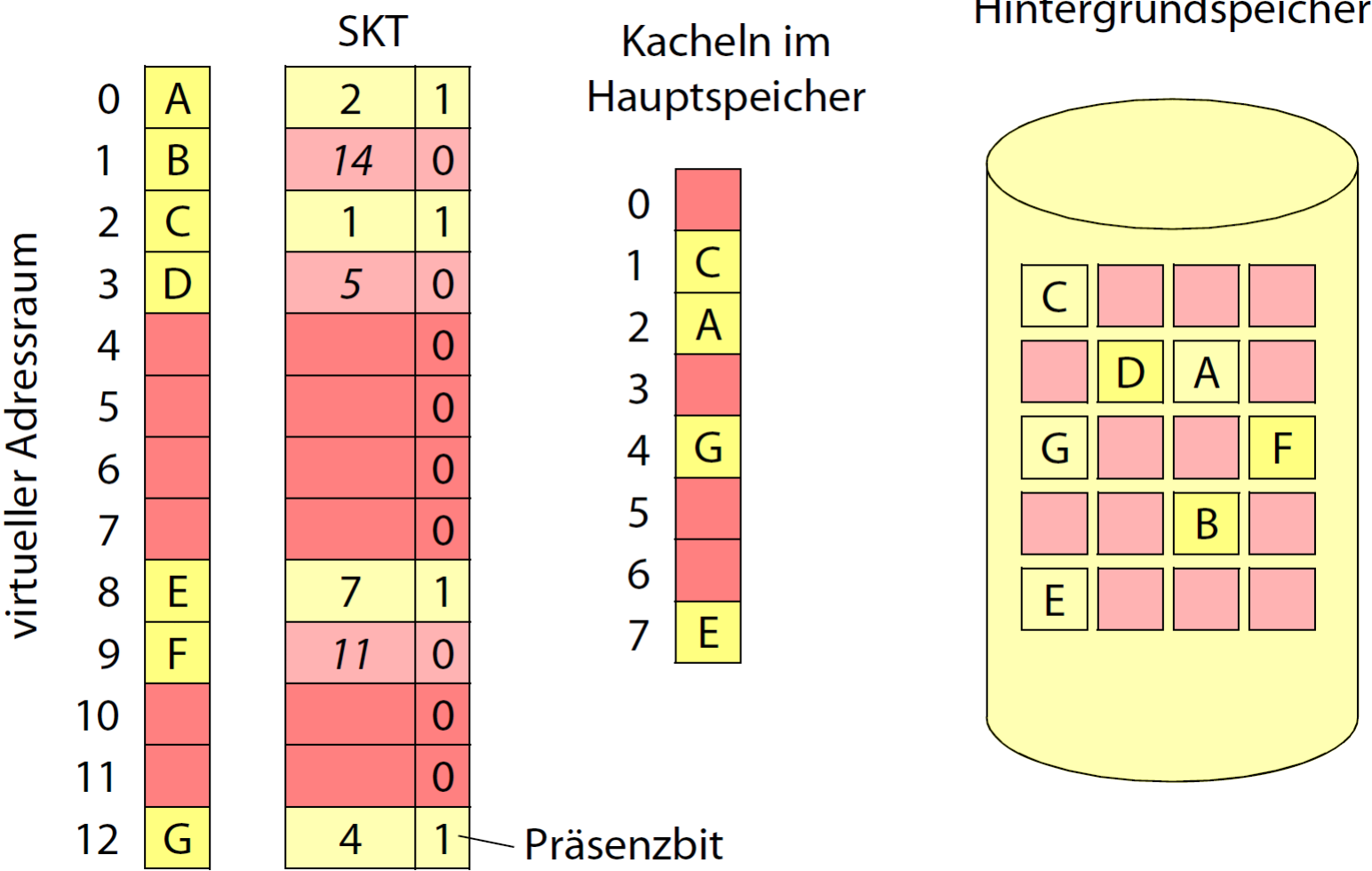
- Prozesse benötigen nicht alle Speicherstellen gleich häufig
 - Bestimmte Befehle werden selten oder gar nicht benutzt (z.B. Fehlerbehandlungen)
 - Bestimmte Datenstrukturen werden nicht voll belegt
- Prozesse benötigen evtl. mehr Speicher als Hauptspeicher vorhanden

Idee

- Vortäuschen eines großen Hauptspeichers
- Einblenden benötigter Speicherbereiche
- Abfangen von Zugriffen auf nicht eingeblendete Bereiche
- Bereitstellen der benötigten Bereiche auf Anforderung
- Auslagern nicht benötigter Bereiche (*Swapping*)

Demand Paging (1)

Bereitstellen von Seiten auf Anforderung



Seitenersetzung

Was tun, wenn keine freie Kachel vorhanden?

- Eine Kachel muss verdrängt werden, um Platz für neue Kachel zu schaffen!
- Auswahl von Kacheln, die nicht geändert wurden (*dirty bit* in der Seitentabelle)
- Verdrängung erfordert Auslagerung, falls Kachel geändert wurde

Vorgang

- Seitenfehler (*page fault*): Unterbrechung
- Auslagern einer Kachel, falls keine freie Kachel verfügbar
- Einlagern der benötigten Kachel
- Wiederholung des Zugriffs

Problem

- Welche Kachel soll ausgewählt werden? → verschiedene Strategien möglich

Freiseitenpuffer

Statt eine Seite zu ersetzen, wird permanent eine Menge freier Seiten gehalten

- Auslagerung geschieht „im Voraus“
- Effizienter: Ersetzungszeit besteht im Wesentlichen nur aus der Einlagerungszeit

Behalten der Seitenzuordnung auch nach der Auslagerung

- Wird die Seite doch noch benutzt, bevor sie durch eine andere ersetzt wird, kann sie mit hoher Effizienz wiederverwendet werden.
- Seite wird aus Freiseitenpuffer ausgetragen und wieder dem entsprechenden Prozess zugeordnet

Seiten werden aus Effizienzgründen nicht gelöscht (genullt).

Seitenadressierung und Mehrprogrammbetrieb (1)

Problem: Verteilung der Kacheln unter mehreren Prozessen

Begrenzungen

- Maximale Kachelmenge: begrenzt durch Speicherausbau eines Rechensystems, d.h. durch Anzahl verfügbarer physikalischer Kacheln
- Minimale Kachelmenge: abhängig von der Prozessorarchitektur
 - Mindestens die Anzahl von Kacheln nötig, die theoretisch maximal bei einem Maschinenbefehl benötigt werden
 - Beispiel:
 - Kachelgröße: 4 Bytes
 - Länge des Befehls: 4 Bytes
 - Länge eines durch den Befehl adressierten Datums: 8 Bytes
 - ☞ Zwei Kacheln für den Befehl, drei Kacheln für die Daten

Deaktivieren von Prozessen (1)

Sind zu viele Prozesse aktiv, werden welche deaktiviert

- Kacheln teilen sich auf weniger Prozesse auf
- Verbindung mit dem *Scheduler* nötig
 - Verhindern von Aushungerung
 - Erzielen kurzer Reaktionszeiten
- Guter Kandidat: Prozess mit wenigen Seiten im Hauptspeicher
 - Geringe Latenz bei Wiedereinlagerung bzw. wenige Seitenfehler bei Aktivierung und *Demand Paging*
- Prozesse haben Prioritäten, welche hier verwendet werden.